

SWI C01 — Reservation System

Start the Engineering Project · splněné povinné výstupy a Definition of Done

Položka	Hodnota
Název týmu	[DOPLNIT]
Členové týmu	Viacheslav Zlobin (sk1drovv) [DOPLNIT zbývající 2-3 členy]
URL repozitáře	https://github.com/sk1drovv/swi-reservation-system
Rezervovaný resource	Nabíjecí stanice pro elektromobily (EV charging station)
Technologický stack	Python 3.12 + FastAPI + SQLAlchemy + PostgreSQL + pytest
Datum	[DOPLNIT]

Kurzívou v hranatých závorkách jsou označena místa, která musí tým doplnit nebo ověřit vlastními daty — zejména jména členů, čísla issue a skutečné výstupy spike.

1 · Tým a repozitář

- Repozitář je veřejný, hostovaný na GitHubu, jeden společný pro celý tým.
- Všichni členové jsou přidáni jako Collaborators (Settings → Collaborators) a mají právo commitovat do main i vytvářet branche.
- Struktura adresářů odpovídá zadání.

```
README.md
docs/
  intent-and-change.md
  architecture-and-decisions.md
  evidence-and-evolution.md
src/
  main.py          # FastAPI app, POST /reservations
  models.py        # Reservation, ChargingStation, User
  schemas.py       # Pydantic validace vstupu
  service.py       # business rules (overlap, denní limit)
  db.py            # připojení k PostgreSQL
tests/
  test_overlap.py  # evidence engineering spike
requirements.txt
.gitignore
```

2 · Reservation domain

Rezervujeme časové sloty na firemních nabíjecích stanicích pro elektromobily. Zaměstnanec si rezervuje okno předem, aby měl po příjezdu jistotu volné stanice.

Povinný prvek	Naše realizace
Resource	ChargingStation — id, connector_type (AC 22 kW / DC 50 kW), power_kw, location, active
Reservation	id, user_id, station_id, start_time, end_time, state, created_at
User	id, email, vehicle_type
States	DRAFT → CONFIRMED → CANCELLED
Operations	create, confirm, cancel, check availability
Common rule	Dvě CONFIRMED rezervace téže stanice se nesmí časově překrývat.

Povinný prvek	Naše realizace
Domain rule	Uživatel smí mít nejvýše jednu CONFIRMED rezervaci za kalendářní den a její délka nesmí přesáhnout 4 hodiny.
Boundary	Notification Service — e-mail při confirm a cancel, volaný přes HTTP; může být pomalý nebo nedostupný.

Zdůvodnění vlastního pravidla

Nabíjecích stanic je málo a poptávka převyšuje kapacitu. Bez limitu by jeden uživatel zablokoval stanici na celý pracovní den. Pravidlo drží dostupnost pro celý tým a zároveň vytváří netriviální validační logiku, kterou lze testovat nezávisle na pravidle o překryvu.

3 · Project Frame

Obsah souboru docs/intent-and-change.md

Reservation domain

Charging slots on company-owned EV charging stations.

Purpose

The system serves employees who drive electric vehicles to work and need a guaranteed charging slot during the working day. It prevents queueing and conflicts at a limited number of stations by reserving time windows in advance.

Users / Stakeholders

- Employee — creates, confirms and cancels their own reservations.
- Facility admin — manages stations and may cancel any reservation.

Core concepts

Reservation, ChargingStation, User, TimeSlot.

Core operations

- Create reservation (result: DRAFT)
- Confirm / approve reservation (DRAFT → CONFIRMED)
- Cancel reservation (DRAFT or CONFIRMED → CANCELLED)
- Check availability of a station within a time range

Persistent state

```
Reservation: id, user_id, station_id, start_time, end_time,
             state, created_at
ChargingStation: id, connector_type, power_kw, location, active
User: id, email, vehicle_type
```

State-changing operation

confirm(reservation_id): DRAFT → CONFIRMED. The overlap check runs at this moment, not at creation time — a DRAFT is only an intent and may safely overlap with other drafts.

Common business rule

Confirmed reservations for the same resource must not overlap.

Domain-specific business rule

A user may hold at most one CONFIRMED reservation per calendar day, and its duration must not exceed 4 hours.

External / system boundary

Notification Service — sends an e-mail when a reservation is confirmed or cancelled. Accessed over HTTP. It is outside our control and may be slow or unavailable.

Assumption

We assume users cancel reservations they no longer need, so no-shows are rare and we do not need an automatic release mechanism yet.

Unknown

We do not know how the system should behave when the Notification Service is unavailable at the moment of confirmation — should the confirmation fail, or should it succeed and notify later?

4 · Selected future pressure

Category	R — Risk / Higher consequence
Concrete pressure	Systém se rozšíří z interního parkoviště na veřejné stanice s placenou rezervací. Dvojitá rezervace pak znamená, že platící zákazník přijede k obsazené stanici.
Why it is relevant	Dnes stojí překryv někoho pět minut čekání. Jakmile do hry vstoupí platba, stejná chyba znamená vracení peněz, stížnosti a ztrátu důvěry. Pravidlo o překryvu proto musí být vynuceno na jednom místě v service vrstvě a pokryto automatickým testem, ne ponecháno na kontrole v UI.

V C01 pressure neimplementujeme — pouze ji fixujeme jako směr, který ovlivní rozhodnutí v dalších týdnech.

5 · Change + review smyčka

Krok	Provedení
Issue	#1 C01 engineering spike: enforce non-overlapping confirmed reservations
Autor změny	[DOPLNIT jméno] — branch spike/overlap-check
Reviewer	[DOPLNIT jméno] — Pull Request #2, review před mergem
Obsah review	Reviewer ověřil, že kontrola překryvu je v service vrstvě (ne v endpointu), že hraniční případ „konec == začátek“ je povolen, a že test skutečně padá bez opravy.
Integrace	PR #2 sloučen do main po schválení reviewerem.

Důkaz v repozitáři: historie Pull Requestu #2 obsahuje komentář reviewera a schválení před mergem.

6 · Engineering spike — varianta A (Persistence)

Zvolena varianta A: rezervace je uložena do skutečné PostgreSQL databáze, znovu načtena a ověřena, včetně vynucení pravidla o překryvu.

Co bylo skutečně spuštěno

```
# spuštění databáze
docker run --name swi-db -e POSTGRES_PASSWORD=swi \
  -e POSTGRES_DB=reservations -p 5432:5432 -d postgres:16

# instalace závislostí a běh testu
pip install -r requirements.txt
pytest tests/test_overlap.py -v
```

Testovaný scénář

- Vytvoření stanice a dvou uživatelů, zápis do DB.
- Rezervace A: 09:00–11:00, confirm → očekáváno CONFIRMED.
- Rezervace B: 10:00–12:00 na téže stanici, confirm → očekávána výjimka `OverlapError`.
- Rezervace C: 11:00–13:00 (navazuje, nepřekrývá se) → očekáváno CONFIRMED.
- Nové načtení ze session ověří, že stav CONFIRMED skutečně přetrval v databázi.

Pozorovaný výsledek

[DOPLNIT skutečný výstup pytest — např. „3 passed in 1.42s“ — a případné odchylky, na které tým narazil.]

7 · Evidence a decision

Obsah souboru docs/evidence-and-evolution.md

C01 Engineering Spike

Question / unknown	Dokáže naše datová vrstva spolehlivě vynutit pravidlo o nepřekrývajících se CONFIRMED rezervacích, nebo se dvě souběžná potvrzení mohou prosadit obě?
What we did	Spustili jsme PostgreSQL v Dockeru, implementovali entity a service vrstvu a napsali test, který se pokouší potvrdit dvě překrývající se rezervace téže stanice a jednu navazující. Test byl skutečně spuštěn příkazem <code>pytest</code> .
Observed result	[DOPLNIT — např.: kontrola v service vrstvě překryv zachytila, test prošel; databáze sama ale žádné omezení nemá, takže při dvou souběžných požadavcích by obě potvrzení mohla projít.]

Decision / what changes	<i>[DOPLNIT — např.: kontrolu v service vrstvě ponecháme jako rychlou validaci, ale do CP1 přidáme databázové omezení (EXCLUDE constraint nad tstzrange), aby pravidlo platilo i při souběhu. Souvisí s vybranou pressure R.]</i>
-------------------------	---

8 · CP1 walking skeleton

Definováno v README.md. Runnable musí být až po C03 / před C04.

```
POST /reservations
→ validate      (Pydantic schema + business rules)
→ persist       (SQLAlchemy → PostgreSQL)
→ return reservation ID
→ automated check (pytest: POST vrátí 201 a ID, záznam je v DB)
```

Cesta je záměrně tenká: jeden endpoint, jedna tabulka, jeden automatický test. Ostatní operace (confirm, cancel, availability) se navěší až na funkční kostru.

9 · Stack decision

Obsah souboru docs/architecture-and-decisions.md

We chose Python 3.12 + FastAPI + SQLAlchemy + PostgreSQL + pytest instead of the default Java / Spring Boot stack. Reason: the team already works with Python, while no member has practical experience with Spring. A familiar stack lets us spend C01 on the reservation domain and on the engineering spike rather than on framework setup.

Trade-off: we support this stack ourselves, without the teacher's reference setup. FastAPI gives us request validation (Pydantic) and an OpenAPI schema out of the box, which covers what the CP1 walking skeleton needs. PostgreSQL is kept from the recommended stack, so the persistence layer stays comparable to other teams.

10 · Definition of Done před C02

#	Položka	Stav
1	tým 3-4 členové	○ <i>doplnit v repu</i>
2	společný repo	✓ splněno
3	jasný reservation domain	✓ splněno
4	Resource + Reservation + User	✓ splněno
5	meaningful Reservation states	✓ splněno
6	create + confirm/approve + cancel + availability	✓ splněno
7	common overlap rule	✓ splněno
8	1 domain-specific business rule	✓ splněno
9	1 external/system boundary	✓ splněno
10	kompletní Project Frame	✓ splněno
11	1 Q/C/R/L future pressure	✓ splněno
12	1 reviewed and integrated change	○ <i>doplnit v repu</i>
13	1 executed engineering spike	○ <i>doplnit v repu</i>
14	spike evidence + decision	○ <i>doplnit v repu</i>
15	definovaný CP1 walking skeleton	✓ splněno

Položky označené „doplnit v repu“ nelze splnit dokumentem — vyžadují skutečnou akci v repozitáři: přidání členů, provedení review před mergem a reálné spuštění spike s výstupem uloženým do docs/evidence-and-evolution.md.